

LAB ACTIVITY (Exp-05)

1. Implement queue operations using Array.

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

void enqueue(int x) {
    if (rear == MAX - 1) {
        printf("Queue Overflow\n");
        return;
    }
    if (front == -1) front = 0;
    queue[++rear] = x;
    printf("%d enqueued\n", x);
}

void dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow\n");
        return;
    }
    printf("%d dequeued\n", queue[front++]);
}

void display() {
    if (front == -1 || front > rear) {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue elements: ");
    for (int i = front; i <= rear; i++)
        printf("%d ", queue[i]);
    printf("\n");
}

int main() {
    int choice, value;
    while (1) {
        printf("\n--- Queue using Array ---\n");
        printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
```

```
switch (choice) {  
    case 1:  
        printf("Enter value to enqueue: ");  
        scanf("%d", &value);  
        enqueue(value);  
        break;  
    case 2:  
        dequeue();  
        break;  
    case 3:  
        display();  
        break;  
    case 4:  
        exit(0);  
    default:  
        printf("Invalid choice!\n");  
}  
}  
return 0;  
}
```

```
--- Queue using Array ---  
1. Enqueue  
2. Dequeue  
3. Display  
4. Exit  
Enter your choice: 1  
Enter value to enqueue: 10  
10 enqueued
```

```
--- Queue using Array ---  
1. Enqueue  
2. Dequeue  
3. Display  
4. Exit  
Enter your choice: 1  
Enter value to enqueue: 20  
20 enqueued
```

```
--- Queue using Array ---  
1. Enqueue  
2. Dequeue  
3. Display  
4. Exit  
Enter your choice: 1  
Enter value to enqueue: 30  
30 enqueued
```

```
--- Queue using Array ---  
1. Enqueue  
2. Dequeue  
3. Display  
4. Exit  
Enter your choice: 2  
10 dequeued
```

```
--- Queue using Array ---  
1. Enqueue  
2. Dequeue  
3. Display  
4. Exit  
Enter your choice: 3  
Queue elements: 20 30
```

2. Implement queue operations using Linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* front = NULL;
struct Node* rear = NULL;

void enqueue(int x) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->data = x;
    temp->next = NULL;

    if (rear == NULL) {
        front = rear = temp;
    } else {
        rear->next = temp;
        rear = temp;
    }
    printf("%d enqueued\n", x);
}

void dequeue() {
    if (front == NULL) {
        printf("Queue Underflow\n");
        return;
    }
    struct Node* temp = front;
    printf("%d dequeued\n", front->data);
    front = front->next;
    if (front == NULL) rear = NULL;
    free(temp);
}

void display() {
    if (front == NULL) {
        printf("Queue is empty\n");
        return;
    }
    struct Node* temp = front;
    printf("Queue elements: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

```
int main() {
    int choice, value;
    while (1) {
        printf("\n--- Queue using Linked List ---\n");
        printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &value);
                enqueue(value);
                break;
            case 2:
                dequeue();
                break;
            case 3:
                display();
                break;
            case 4:
                exit(0);
            default:
                printf("Invalid choice!\n");
        }
    }
    return 0;
}
```

--- Queue using Linked List ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter value to enqueue: 10

10 enqueued

--- Queue using Linked List ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter value to enqueue: 20

20 enqueued

--- Queue using Linked List ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter value to enqueue: 30

30 enqueued

--- Queue using Linked List ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 2

10 dequeued

--- Queue using Linked List ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 3

Queue elements: 20 30

--- Queue using Linked List ---

1. Enqueue

3. Implement circular queue using array and linked list

1. Using Array

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

void enqueue(int x) {
    if ((rear + 1) % MAX == front) {
        printf("Circular Queue Overflow\n");
        return;
    }
    if (front == -1) {
        front = rear = 0;
    } else {
        rear = (rear + 1) % MAX;
    }
    queue[rear] = x;
    printf("%d enqueued\n", x);
}

void dequeue() {
    if (front == -1) {
        printf("Circular Queue Underflow\n");
        return;
    }
    int x = queue[front];
    if (front == rear) {
        front = rear = -1;
    } else {
        front = (front + 1) % MAX;
    }
    printf("%d dequeued\n", x);
}

void display() {
    if (front == -1) {
        printf("Circular Queue is empty\n");
        return;
    }
    printf("Circular Queue elements: ");
    int i = front;
    while (1) {
        printf("%d ", queue[i]);
        if (i == rear) break;
        i = (i + 1) % MAX;
    }
    printf("\n");
}
```

```
int main() {
    int choice, value;
    while (1) {
        printf("\n--- Circular Queue using Array ---\n");
        printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &value);
                enqueue(value);
                break;
            case 2:
                dequeue();
                break;
            case 3:
                display();
                break;
            case 4:
                exit(0);
            default:
                printf("Invalid choice!\n");
        }
    }
    return 0;
}
```

--- Circular Queue using Array ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter value to enqueue: 10

10 enqueued

--- Circular Queue using Array ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter value to enqueue: 20

20 enqueued

--- Circular Queue using Array ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter value to enqueue: 30

30 enqueued

--- Circular Queue using Array ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 2

10 dequeued

--- Circular Queue using Array ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 3

Circular Queue elements: 20 30

--- Circular Queue using Array ---

1. Enqueue
2. Dequeue

2. Using Linked List

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* front = NULL;
struct Node* rear = NULL;

int size = 0;
int N = 5;

void enqueue(int x) {
    if (size == N) {
        printf("Circular Queue Overflow\n");
        return;
    }

    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->data = x;
    temp->next = NULL;

    if (front == NULL) {
        front = rear = temp;
        rear->next = front;
    } else {
        rear->next = temp;
        rear = temp;
        rear->next = front;
    }

    size = (size + 1) % (N + 1);
    printf("%d enqueued\n", x);
}

void dequeue() {
    if (front == NULL) {
        printf("Circular Queue Underflow\n");
        return;
    }

    int x;
    if (front == rear) {
```

```

        x = front->data;
        free(front);
        front = rear = NULL;
    } else {
        struct Node* temp = front;
        x = temp->data;
        front = front->next;
        rear->next = front;
        free(temp);
    }

    size--;
    printf("%d dequeued\n", x);
}

void display() {
    if (front == NULL) {
        printf("Circular Queue is empty\n");
        return;
    }

    printf("Circular Queue elements: ");
    struct Node* temp = front;
    do {
        printf("%d ", temp->data);
        temp = temp->next;
    } while (temp != front);
    printf("\n");
}

int main() {
    int choice, value;
    while (1) {
        printf("\n--- Circular Queue using Linked List ---\n");
        printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &value);
                enqueue(value);
                break;
            case 2:
                dequeue();
                break;

```

```
        case 3:
            display();
            break;
        case 4:
            exit(0);
        default:
            printf("Invalid choice!\n");
    }
}
return 0;
}
```

--- Circular Queue using Linked List ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 1

Enter value to enqueue: 10

10 enqueued

--- Circular Queue using Linked List ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 1

Enter value to enqueue: 20

20 enqueued

--- Circular Queue using Linked List ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 1

Enter value to enqueue: 30

30 enqueued

--- Circular Queue using Linked List ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 2

10 dequeued

--- Circular Queue using Linked List ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 3

Circular Queue elements: 20 30

--- Circular Queue using Linked List ---

1. Enqueue